

Proyecto Programado CI-0123

Armador de figuritas con piezas plásticas (Lego®)

Equipo 4
Agustín Soto, Brandon Alfaro,
Joaquín, Enrique

Protocolo Grupal-Uppercase v3.0

En este documento se explica el protocolo de comunicación entre los elementos del proyecto. Se tiene un cliente, un intermediario (fork) y un servidor. El cliente únicamente se comunica con el fork, quien a su vez se comunica con el servidor. El servidor se comunica con el fork, que transmite sus mensajes al cliente. Varios clientes y servidores se pueden conectar al mismo fork, por lo que este crea un hilo para manejar independientemente cada conexión.

1. Descripción

El protocolo tiene 3 parámetros separados por el carácter /. Se llama “Uppercase” ya que sus dos primeros parámetros sólo utilizan una letra mayúscula para indicar el tipo de mensaje a transmitir. Los mensajes siguen una estructura estándar con el siguiente formato:

[Protocolo]/[Comando]/[mensaje]

Donde **Protocolo** siempre es la letra P, su única función es indicar que se está utilizando este protocolo. **Comando** es otra letra mayúscula que indica la función a realizar. El tercer parámetro es un mensaje opcional, se utiliza para enviar datos o información. En algunos casos, como en un cierre de conexión, mensaje puede omitirse.

Como se tiene la restricción de ser 3 parámetros y de que los dos primeros sean solo una letra, entonces se sabe que los primeros 4 caracteres son parte del protocolo y el resto es parte del mensaje. Esto permite que se pueda utilizar P/ dentro del mensaje sin generar problemas.

Ventajas:

- **Implementación:** la construcción, procesamiento y limpieza del protocolo conllevan métodos fáciles de realizar.
- **User Friendly:** por su sencillez, cualquier usuario puede aprender y utilizarlo rápidamente.

2. Comandos

Nombre	Representación	Descripción
ACK	A	Indica que el comando se hizo correctamente.
Connect	C	Crear conexión inicial fork-server.
Data	D	Respuesta del server, como directorios o figuras.
Fork	F	Comunicación entre Forks.
Get	G	Solicitud de figuras.
Kill	K	Servidor anuncia su muerte.
Patch	P	Actualización de servidores.
Quit	Q	Cerrar conexión fork-server.
Request	R	Solicitud de servidores o directorios.
Server Data	S	Dirección IP y puerto del servidor.

Cuadro 1: Comandos del Protocolo

3. Flujo de Conexión

3.1. Descubrimiento de Servidores

Primero se levanta el intermediario, este crea un proceso secundario para realizar broadcast a la dirección 172.16.123.79 preguntando por servidores disponibles con el siguiente mensaje:

```
Fork > P/C/172.16.123.70:8080* > Broadcast
```

Cuando se levanta un servidor escucha por broadcast, recibirá el mensaje anterior con lo que obtendrá la dirección y puerto del intermediario. Ahora el servidor establece una conexión TCP con el intermediario y le envía sus propios datos:

```
Server > P/S/172.16.123.71:8080 > Fork
```

El intermediario crea otro proceso secundario para atender cada conexión TCP. Recibe los datos del servidor y los agrega en una estructura de datos compartida entre el resto de procesos para que funcione como mapa de servidores.

3.2. Comunicación con Cliente

El cliente conoce la dirección IP y puerto del intermediario. Cuando inicia, establece una conexión TCP. El cliente puede solicitar el directorio de figuras, una figura en particular o terminar la conexión.

```
// Solicitud a intermediario
Client > P/R/dir > Fork

// Para cada servidor
Fork > P/R/dir > Server
Server > P/D/fig1, fig2, fig3 > Fork

// Respuesta a cliente
Fork > P/D/fig1, fig2, fig3 > Client
```

Una vez tiene el listado de figuras, el cliente puede solicitar alguna:

```
// Solicitud a intermediario
Client > P/G/fig1 > Fork

// Envía solicitud a servidor
Fork > P/G/fig1 > Server
Server > P/D/brown block 1, red block... > Fork

// Respuesta a cliente
Fork > P/D/brown block 1, red block > Client
```

Para terminar la conexión utiliza el comando QUIT:

```
Client > P/Q/ > Fork
```

3.3. Comunicación con Cliente Web

Todos los elementos tienen la capacidad de codificar y decodificar el protocolo de comunicación. A la hora de decodificar un mensaje HTTP, el programa detecta que no se está utilizando el protocolo e inicia un procedimiento para manejar HTTP.

Si el servidor recibe el mensaje, extrae la solicitud, obtiene el nombre de figura, la lee de su sistema de archivos y la agrega a una respuesta HTTP. Si el intermediario recibe la solicitud, detecta que no se sigue el protocolo, extrae el nombre de la figura, identifica el servidor correspondiente y reenvía el mensaje.

3.4. Notas Generales

- Las direcciones IPv4 utilizadas son las asignadas al equipo 4 del grupo 3 del laboratorio 3-5.
- De momento no se utiliza el comando ACK durante el flujo de conexión.

3.5. Direcciones DHCP asignables por isla

- Isla 1: 172.16.123.[19-30] > BC 172.16.123.31
- Isla 2: 172.16.123.[35-46] > BC 172.16.123.47
- Isla 3: 172.16.123.[51-62] > BC 172.16.123.63
- Isla 4: 172.16.123.[67-78] > BC 172.16.123.79
- Isla 5: 172.16.123.[83-94] > BC 172.16.123.95
- Isla 6: 172.16.123.[99-110] > BC 172.16.123.111

3.6. Mapa de las VLAN/IP

- Isla 1: VLAN 610, IP 172.16.123.16/28, IP L3 .17, IP L2 .18
- Isla 2: VLAN 620, IP 172.16.123.32/28, IP L3 .33, IP L2 .34
- Isla 3: VLAN 630, IP 172.16.123.48/28, IP L3 .49, IP L2 .50
- Isla 4: VLAN 640, IP 172.16.123.64/28, IP L3 .65, IP L2 .66
- Isla 5: VLAN 650, IP 172.16.123.80/28, IP L3 .81, IP L2 .82
- Isla 6: VLAN 660, IP 172.16.123.96/28, IP L3 .97, IP L2 .98

4. Propuesta para comunicación I a I y “Muerte” de servidores

La propuesta para comunicación entre islas es usar broadcast para avisar a los demás forks de las figuras que se encuentran en su servidor designado.

Fork > P/F/Fig1, Fig2, Fig3 > Broadcast para vlan de forks

El comando F/ es exclusivo de los forks y avisa la lista de figuras asignadas a una IP específica. En caso de cambios por entrada o muerte de servidores, se puede usar este mismo comando para reescribir la lista de figuras asignadas.

Para la muerte de un servidor se recomienda usar el comando K/:

Server > P/K/ > Fork

Fork > P/F/ > Broadcast para vlan de forks

Si el servidor se desconecta sin avisar, el fork debería usar un sistema de timeout para determinar si el servidor está caído.

5. Casos de Prueba

N.	Caso de prueba	Entrada del cliente	Respuesta esperada	Resultado esperado
1	Solicitar lista de figuras disponibles	P/R/dir	P/D/fig1, fig2, fig3	El cliente recibe la lista de figuras disponibles.
2	Solicitar figura válida	P/G/elefante	P/D/pieza1,2, pieza2,4	El cliente recibe las piezas necesarias para construir la figura solicitada.
3	Solicitar otra figura válida	P/G/mapache	P/D/ojo,2, nariz,1, bloque,4	El cliente recibe correctamente el inventario de piezas de la figura.
4	Solicitar figura inexistente	P/G/dragon	P/D/404	El cliente recibe un mensaje indicando que la figura no fue encontrada.
5	Solicitar directorio cuando no hay servidores disponibles	P/R/dir	P/S/	El cliente recibe una respuesta vacía o indicación de que no hay servidores registrados.
6	Enviar solicitud con formato incorrecto	P/G/	P/D/400	El cliente recibe una advertencia o error por solicitud inválida.
7	Enviar comando desconocido	P/X/fig1	P/D/400	El sistema rechaza el comando porque no pertenece a la tabla oficial del protocolo.
8	Cerrar conexión correctamente	P/Q/	Cierre de socket	El cliente termina la conexión con el intermediario y finaliza el programa.
9	Registrar servidor ante el intermediario	P/S/172.16.123.71:8080	Registro en mapa de servidores	El intermediario almacena la IP y puerto del servidor disponible.
10	Servidor anuncia desconexión	P/K/	Actualización del mapa de servidores	El intermediario elimina o marca como no disponibles las figuras asociadas al servidor.

Cuadro 2: Casos de prueba adaptados al protocolo Uppercase